Lab Exercise 17- Creating Secrets in Kubernetes

Creating secrets in Kubernetes is an essential practice for managing sensitive information such as passwords, OAuth tokens, and SSH keys. Let's go through a lab exercise that demonstrates how to create different types of secrets in Kubernetes.

Step 1: Create a Generic Secret

Create a secret from literal values:

```
kubectl create secret generic my-secret --from-literal=username=admin --from-literal=password='P@ssw0rd'
```

```
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB % kubectl create secret generic my-secret --from-literal
=username=admin --from-literal=password='P@ssw0rd'

secret/my-secret created
```

Verify the secret:

```
kubectl get secrets my-secret -o yaml
```

```
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB % kubectl get secrets my-secret -o yaml

apiVersion: v1
data:
  password: UEBzc3cwcmQ=
  username: YWRtaW4=
kind: Secret
metadata:
  creationTimestamp: "2026-03-16T05:11:47Z"
  name: my-secret
  namespace: default
  resourceVersion: "16218"
  uid: 94b652f6-5c40-417d-bcbb-eb8ca6ee2154
type: Opaque
```

The output will be base64 encoded. To decode, you can use:

```
echo -n “P@ssword” | base64
```

```
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB % echo -n “P@ssw0rd” | base64

4oCcUEBzc3cwcmTigJ0=
```

Step 2: Create a Secret from a File

Create a file with your sensitive data:

```
echo -n 'my-username' > ./username.txt



echo -n 'my-password' > ./password.txt
```

```
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB % echo -n 'my-username' > ./username.txt


echo -n 'my-password' > ./password.txt
```

Create a secret from the file:

```
kubectl create secret generic my-file-secret --from-file=username=./username.txt --from-file=password=./password.txt
```

```
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB % kubectl create secret generic my-file-secret --from-file=username=./username.txt --from-file=password=./password.txt
secret/my-file-secret created
```

Verify the secret:

```
kubectl get secrets my-file-secret -o yaml
```

```
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB % kubectl get secrets my-file-secret -o yaml

apiVersion: v1
data:
  password: bXktcGFzc3dvcmQ=
  username: bXktdXNlcm5hbWU=
kind: Secret
metadata:
  creationTimestamp: "2026-03-16T05:12:27Z"
  name: my-file-secret
  namespace: default
  resourceVersion: "16287"
  uid: 77e518d3-f300-4e7f-a773-95f1e523f89d
type: Opaque
```

Step 3: Create a Docker Registry Secret

Create a Docker registry secret:

```
kubectl create secret docker-registry my-registry-secret --docker-username=<your-username> --docker-password=<your-password> --docker-email=<your-email> --docker-server=<your-docker-server>
```

```
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB % kubectl create secret docker-registry my-registry-secr
et \
--docker-username=myuser \
--docker-password=mypassword \
--docker-email=myemail@example.com \
--docker-server=https://index.docker.io/v1/
secret/my-registry-secret created
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB % kubectl get secrets
NAME                 TYPE                             DATA   AGE
my-file-secret       Opaque                           2      4m54s
my-registry-secret   kubernetes.io/dockerconfigjson   1      5s
my-secret            Opaque                           2      5m34s
```

Verify the secret:

```
kubectl get secrets my-registry-secret -o yaml
```

```
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB % kubectl get secret my-registry-secret -o yaml
apiVersion: v1
data:
  .dockerconfigjson: eyJhdXRocyI6eyJodHRwczovL2luZGV4LmRvY2tlci5pby92MS8iOnsidXNlcm5hbWUiOiJteXVzZXIiLCJwY
XNzd29yZCI6Im15cGFzc3dvcmQiLCJlbWFpbCI6Im15ZW1haWxAZXhhbXBsZS5jb20iLCJhdXRoIjoiYlhsMWMyVnlPbTE1Y0dGemMzZHZ
jbVE9In19fQ==
kind: Secret
metadata:
  creationTimestamp: "2026-03-16T05:17:16Z"
  name: my-registry-secret
  namespace: default
  resourceVersion: "16603"
  uid: d29d8169-609c-49a1-b25f-fb1b5d03497b
type: kubernetes.io/dockerconfigjson
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB %
```

Step 4: Create a Secret Using a YAML Manifest

Create a YAML file for the secret:

Create a file named **my-secret.yaml** with the following content:

```
apiVersion: v1
kind: Secret
metadata:
  name: my-manifest-secret
type: Opaque
data:
  username: YWRtaW4=    # base64 encoded value of 'admin'
  password: UEBzc3cwcmQ=  # base64 encoded value of 'P@ssw0rd'
```

```
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB % cat my-secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: my-manifest-secret
type: Opaque
data:
  username: YWRtaW4=
  password: UEBzc3cwcmQ=
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB %
```

Apply the YAML file:

```
kubectl apply -f my-manifest-secret.yaml
```

```
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB % kubectl apply -f my-secret.yaml

secret/my-manifest-secret unchanged
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB %
```

Verify the secret:

```
kubectl get secrets my-manifest-secret -o yaml
```

```
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB % kubectl get secrets my-secret -o yaml

apiVersion: v1
data:
  password: UEBzc3cwcmQ=
  username: YWRtaW4=
kind: Secret
metadata:
  creationTimestamp: "2026-03-16T05:11:47Z"
  name: my-secret
  namespace: default
  resourceVersion: "16218"
  uid: 94b652f6-5c40-417d-bcbb-eb8ca6ee2154
type: Opaque
```

Step 5: Use Secrets in Pods

Create a pod that uses the secret:

Create a file named **pod-with-secret.yaml** with the following content:

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-secret
spec:
  containers:
  - name: mycontainer
    image: nginx
    env:
    - name: USERNAME
      valueFrom:
        secretKeyRef:
          name: my-secret
          key: username
    - name: PASSWORD
```

```
    valueFrom:
      secretKeyRef:
        name: my-secret
        key: password
```

```
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB % cat pod-with-secret.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-secret
spec:
  containers:
  - name: mycontainer
    image: nginx
    env:
    - name: USERNAME
      valueFrom:
        secretKeyRef:
          name: my-manifest-secret
          key: username
    - name: PASSWORD
      valueFrom:
        secretKeyRef:
          name: my-manifest-secret
          key: password
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB %
```

Apply the YAML file:

```
kubectl apply -f pod-with-secret.yaml
```

```
          key: password
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB % kubectl apply -f pod-with-secret.yaml

pod/pod-with-secret unchanged
```

Verify the pod is running and the secret is being used:

```
kubectl exec pod-with-secret -- printenv | grep -E 'USERNAME|PASSWORD'
```

```
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB % kubectl exec pod-with-secret -c mycontainer -- printen
v | grep -E 'USERNAME|PASSWORD'

USERNAME=admin
PASSWORD=password123
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB %
```

Step 7: Clean Up

Delete the secrets:

```
kubectl delete secret my-secret
kubectl delete secret my-file-secret
kubectl delete secret my-registry-secret
kubectl delete secret my-manifest-secret
```

```
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB % kubectl delete secret my-secret
kubectl delete secret my-file-secret
kubectl delete secret my-registry-secret
kubectl delete secret my-manifest-secret
secret "my-secret" deleted from default namespace
secret "my-file-secret" deleted from default namespace
secret "my-registry-secret" deleted from default namespace
secret "my-manifest-secret" deleted from default namespace
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB %
```

Delete the pod:

```
kubectl delete pod pod-with-secret
```

```
shivanyabharadwaj@Shivanyas-MacBook-Pro DevOpsLAB % kubectl delete pod pod-with-secret
pod "pod-with-secret" deleted from default namespace
```